Permission-Aware Tool Interfaces for Safe ReAct Agents

Abstract

ReAct agents gain powerful operational capabilities through external tool invocation, but their safety fundamentally depends on the behavior of the underlying LLM. This creates a critical vulnerability: jailbreaks, hallucinated tool calls, or incorrect reasoning may trigger high-privilege actions.
This paper addresses this issue not by constraining the LLM, but by modifying the **structure of the tool interface itself**, introducing **Permission-Aware Tool Interfaces (PATI)**.

In PATI, each tool declares a `required_permission_level`, and the agent is initialized with a fixed `user_permission_level` supplied by an external system. This permission value is forcibly injected into every tool call. As a result, regardless of what the LLM outputs, **tools beyond the user's privilege level become structurally impossible to execute**.
This approach is robust against jailbreaks and hallucinations, adheres to the principle of least privilege, preserves model performance, and can be integrated into existing ReAct / LangGraph pipelines with minimal modification.

This work proposes a new design principle:
safety should be enforced at the I/O boundary, not inside the LLM.
By shifting the safety boundary to the tool interface layer, PATI provides a model-independent and structurally reliable method for mitigating external-action risks in ReAct agents.

---

1. Introduction

Recent advances in large language models (LLMs) have intensified public and academic debate over whether AI systems may pose serious risks to society. These discussions often frame **intelligence itself as inherently dangerous**, emphasizing the model's reasoning ability or potential for self-improvement as sources of risk.

However, **no matter how capable an LLM becomes, it cannot exert real-world influence unless it can execute external tools**.
An intelligence that cannot act on the external environment has no social or physical impact; its outputs remain confined to text.
Therefore:

> **Most practical risks of AI do not originate inside the model,**
> **but from whether the system can act on the world through external tools.**

This point is especially critical for tool-enabled agents such as ReAct.
ReAct allows an LLM to autonomously select and invoke external tools during its reasoning process—API calls, file operations, web access, and more.
As a result, several structural risks emerge:

- hallucinated tool calls directly translate into external actions
- jailbreaks may trigger high-privilege operations

- tool permissions are often ambiguous, violating the principle of least privilege
- safety depends heavily on prompt-level control, which is fragile and easily bypassed

These issues arise **not because the model's intelligence is dangerous**, but because **the tool-execution layer is unconstrained**.

This paper corrects this structural misunderstanding in AI-risk discourse and argues that
the safety boundary should reside not inside the LLM, but at the interface between the LLM and external tools.

To address this, we propose a new architecture for fundamentally improving the safety of ReAct agents:
embedding a "permission level" as a mandatory argument in every tool API, and enforcing execution constraints based on the user's privilege level.

This approach provides:

- robustness to hallucinations
- robustness to jailbreaks
- automatic enforcement of least-privilege
- no degradation of model performance

Together, these properties form a structurally strong safety foundation.

Importantly, this work targets **model-induced external-action risks**—cases where an LLM unintentionally executes dangerous operations.
It does **not** attempt to solve all safety problems.
In particular, risks arising from:

- misconfigured permissions
- malicious users intentionally abusing high-privilege tools
- organizational governance failures

are **human-origin risks** and fall outside the scope of this method.

The goal of this paper is to show that
model-origin risks can be structurally mitigated by redesigning the tool-interface layer,
while human-origin risks require separate operational and governance measures.

---

2. Problem Statement

2.1 ReAct Agents and the Tool Execution Problem

ReAct agents operate through a loop in which the LLM selects external tools during its reasoning process and incorporates their outputs into subsequent reasoning steps:

1. The LLM observes and reasons
2. It selects a tool
3. The tool is executed
4. The result is fed back into the next reasoning step

This design is flexible and powerful, but it has a critical structural property:
tool execution depends entirely on the LLM's output.
As a result, several risks arise:

- **Incorrect reasoning → incorrect external actions**
- **LLM drift or runaway behavior → misuse of high-privilege tools**
- **Prompt jailbreak → safety constraints bypassed**
- **No concept of permissions → any tool can be executed**

In other words:

> **The danger of ReAct does not stem from the LLM's intelligence,**
> **but from the *unrestricted tool-execution pathway*.**

No matter how capable an LLM is, it has zero real-world impact if it cannot execute external tools.
Conversely, if tool execution is unrestricted, even small reasoning errors can be amplified into significant external actions.
This ambiguity in the "boundary of influence" is the core structural risk of ReAct.

2.2 Existing Mitigations and Their Limitations

Current ReAct-style agents rely on several mitigation strategies:

- instructing the model via prompts not to perform dangerous actions
- filtering or post-processing the LLM's tool-call outputs
- strengthening model-side alignment

However, these are **not structural safety mechanisms**.

(1) Prompt-based control is fragile
Jailbreaks can easily override or disable prompt-level constraints.

(2) Output filtering is incomplete
It requires accurately interpreting the intent of tool calls,
and is prone to false positives and false negatives.

(3) Model alignment cannot control external actions
Alignment governs *what the model says*,
but **cannot govern *what the system is allowed to execute***.

Thus, existing mitigations rely on the LLM's "good behavior" and
do not structurally prevent unsafe external actions.

2.3 Scope of This Problem (and What It Is Not)

This paper focuses on:

> **model-origin risks in which an LLM unintentionally executes dangerous external tools.**

In contrast, the following **human-origin risks** are outside the scope of this work:

- intentional misuse by high-privilege users
- misconfigured or overly broad permissions
- organizational governance failures
- security vulnerabilities in external APIs
- harmful content generation (content-safety issues)

These require separate operational or governance measures and are not addressed here.

The focus of this work is:

> **to structurally suppress the ReAct-specific problem**
> **where LLM misjudgments translate into external actions,**
> **by redesigning the tool-interface layer.**

2.4 Core Insight

Based on the above, the safety problem of ReAct can be summarized as follows:

- the danger lies not in the LLM's intelligence, but in the pathway to external action
- existing mitigations are model-centric and non-structural
- the true safety boundary lies **not inside the model, but in the tool-execution layer**

This insight forms the foundation of the method proposed in this paper.

---

3. Related Work

Research on LLM safety can be broadly categorized into three areas.

3.1 Model-Centric Safety (Alignment, RLHF, Guardrails)

Traditional AI-safety research has focused primarily on controlling **the internal behavior of the model**.
Techniques such as RLHF and Constitutional AI aim to prevent LLMs from generating harmful, dangerous, or inappropriate content.
Guardrails and jailbreak-prevention methods attempt to suppress unintended responses caused by prompt manipulation.

However, these approaches regulate **what the model says**,
not **what the model is allowed to execute**.
For agents that can invoke external tools, content safety and action safety are fundamentally different problems; strengthening the former does not guarantee the latter.

3.2 Agents and Tool Execution (ReAct, Toolformer, LangChain)

ReAct (Reason + Act) is widely used as a framework in which an LLM selects external tools during its reasoning process and incorporates their results into subsequent steps.
Implementations such as LangChain and LangGraph extend this structure, enabling multi-tool coordination and autonomous task execution.

However, these frameworks **lack any notion of permission management**,
operating under the assumption that the LLM may freely invoke any available tool.
This leaves a structural vulnerability: hallucinations or jailbreaks can directly translate into unsafe external actions.

Research such as Toolformer explores learning tool-use capabilities,
but it likewise **does not address tool safety or permission control**.

3.3 API Redesign Discussions (Agent-First APIs and Beyond)

Recent work has raised the concern that
APIs designed for humans are not suitable for autonomous agents.
Agent-First API (2025), for example, argues that agent-oriented APIs should follow different design principles and acknowledges the need for permission management.

However, these discussions remain largely conceptual and do not propose concrete mechanisms.
In particular, no prior work has introduced a structural safety measure in which:

- **permission_level is embedded as a mandatory argument in every tool API**, and
- **the tool cannot execute unless the LLM provides a valid permission value**.

This specific architectural constraint is absent from existing literature.

3.4 Positioning of This Work

As shown above, existing research focuses on model-internal safety or improving agent capabilities,
but **no prior work provides a structural, API-level mechanism for ensuring the safety of external actions**.

This paper proposes a fundamentally different approach:
embedding permission levels directly into the tool interface,
thereby structurally suppressing external-action risks in ReAct agents.
This distinguishes our work from prior research and provides a practical, model-independent safety mechanism.

---

4. Proposed Method: Permission-Aware Tool Interfaces

To structurally suppress external-action risks in ReAct agents, this work proposes an architecture in which **every tool API embeds a mandatory `permission_level` argument, and tool execution is forcibly constrained based on the user's privilege level**.

The core idea is simple:

> **Safety should be enforced at the tool layer, not inside the LLM.**

4.1 Introducing a Fixed `user_permission_level`

In the proposed method, the ReAct agent is initialized with a
fixed `user_permission_level` obtained from an external system (e.g., RBAC / IAM):

```
user_permission_level = 2
agent = ReActAgent(
    model=LLM(...),
    permission=user_permission_level
)
```

Crucially:

- **the LLM does not generate this value**
- **the LLM cannot modify this value**
- **the value is determined externally and injected into the agent**

Thus, the LLM merely
passes the externally provided permission value to each tool call,
making privilege forgery impossible.

This design ensures that
permission enforcement occurs outside the LLM (out-of-band enforcement).

4.2 Tool-Level Permissions

Each tool declares a `required_permission_level`, for example:

```
delete_file:
    required_permission_level = 3

send_email:
    required_permission_level = 2

search_web:
    required_permission_level = 1
```

Every tool API must follow the form:

```
tool_name(args..., user_permission_level)
```

Inside the tool:

```
if user_permission_level < required_permission_level:
    return "Permission denied"
```

This guarantees that:

- even if the LLM behaves incorrectly
- even if the model is jailbroken
- even if prompts are modified

the tool itself rejects unauthorized execution,
preventing unsafe external actions.

4.3 Implementability in LangGraph / LangChain

The tool-execution pipeline in LangGraph / LangChain consists of three stages:

1. **The LLM generates a tool-call string**
   Example:

   ```
   delete_file(path="foo.txt")
   ```

2. **LangGraph parses the string and maps it to a Python function**
   → At this stage, **the external `user_permission_level` can be injected**

3. **The Python tool function is executed**
   → Permission checks occur here

This means the framework naturally supports injecting additional arguments beyond those produced by the LLM.

Example Execution Flow

LLM output:

```
delete_file(path="foo.txt")
```

LangGraph internal processing:

```
parsed = {"path": "foo.txt"}
parsed["user_permission_level"] = agent.permission  # inject external value
```

Actual tool execution:

```
delete_file(path="foo.txt", user_permission_level=2)
```

Tool logic:

```
if user_permission_level < 3:
    return "Permission denied"
```

→ **Safe by construction.**

Thus, the proposed method aligns cleanly with existing frameworks and is easy to implement.

4.4 The Permission Layer

The architecture consists of three layers:

(1) LLM (ReAct Layer)

- performs observation and reasoning
- proposes tool calls
- passes `user_permission_level` (but does not generate it)

(2) Permission Layer (Core of the Proposal)

- tool APIs require a permission argument
- LangGraph injects the external value
- unauthorized calls are rejected immediately

(3) Tool Execution Layer

- only authorized operations are executed
- all external actions become permission-aware

4.5 Redefining the Action Space of ReAct

Traditional ReAct:

```
AllowedTools = all tools
```

Proposed method:

```
AllowedTools = { t | user_permission_level >= t.required_permission_level }
```

Thus, **the action space itself becomes structurally constrained by permissions**.

Consequences:

- incorrect LLM reasoning cannot trigger high-privilege tools
- jailbreaks cannot escalate privileges
- least-privilege is automatically enforced

4.6 Implementation Sketch

Example Tool Definition

```
def delete_file(path, user_permission_level):
    required = 3
    if user_permission_level < required:
        return "Permission denied"
    ...
```

Integration into the ReAct Prompt

The LLM is provided with:

- the fixed `user_permission_level`
- the list of available tools
- each tool's `required_permission_level`

The LLM chooses actions within these constraints.

---

5. System Architecture

This section presents the overall structure of the proposed method, Permission-Aware Tool Interfaces.
The architecture is designed to **guarantee the safety of external actions without restricting the reasoning capabilities of the LLM**.

5.1 Overview

The proposed architecture consists of three layers:

Layer 1: LLM (ReAct Reasoning Layer)

- Performs observation and reasoning
- Proposes tool calls
- Uses a **fixed** `user_permission_level` **provided externally**, not generated by the LLM
- The LLM's role is limited to generating candidate actions

Layer 2: Permission Layer (Core of the Proposal)

- Hooks into the tool-parsing mechanism of LangGraph / LangChain
- Injects the externally provided `user_permission_level` into every tool call
- Compares it with the tool's `required_permission_level`
- **Rejects unauthorized calls immediately**
- Blocks LLM drift, jailbreaks, and incorrect reasoning at this layer

---

Layer 3: Tool Execution Layer

- Executes actual APIs, file operations, web access, etc.
- Every tool follows the form:

```
tool_name(args..., user_permission_level)
```

- The tool checks its `required_permission_level` internally
- **Execution occurs only if the permission requirement is satisfied**

5.2 Data Flow

The following describes the flow used in the architecture diagram.

Step 1: LLM Reasoning

```
User Input → LLM → "delete_file(path='foo.txt')"
```

The LLM generates a tool-call string,
but **does not generate any permission value**.

Step 2: Permission Injection (LangGraph)

LangGraph parses the LLM output and injects the external permission value:

```
Parsed Call:
{
  "path": "foo.txt",
  "user_permission_level": agent.permission  # external value
}
```

Step 3: Permission Check (Tool Side)

Inside the tool:

```
if user_permission_level < required_permission_level:
    return "Permission denied"
```

Step 4: Tool Execution

Only if the permission requirement is satisfied:

```
delete_file("foo.txt") → executed
```

5.3 Behavior of ReAct Under Permission Constraints

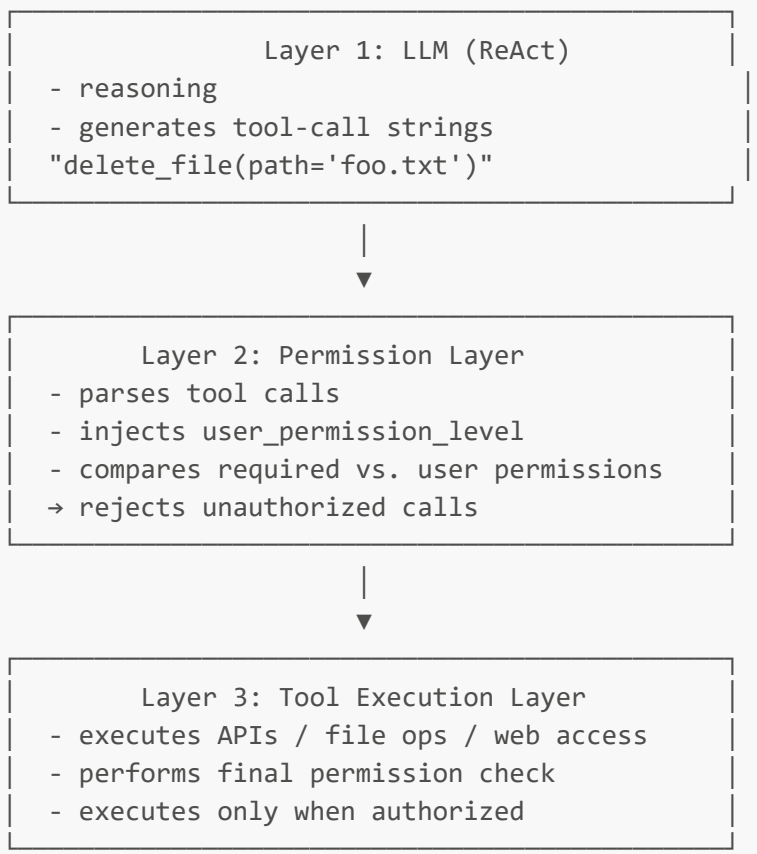
Under the proposed method, the action space of ReAct is redefined as:

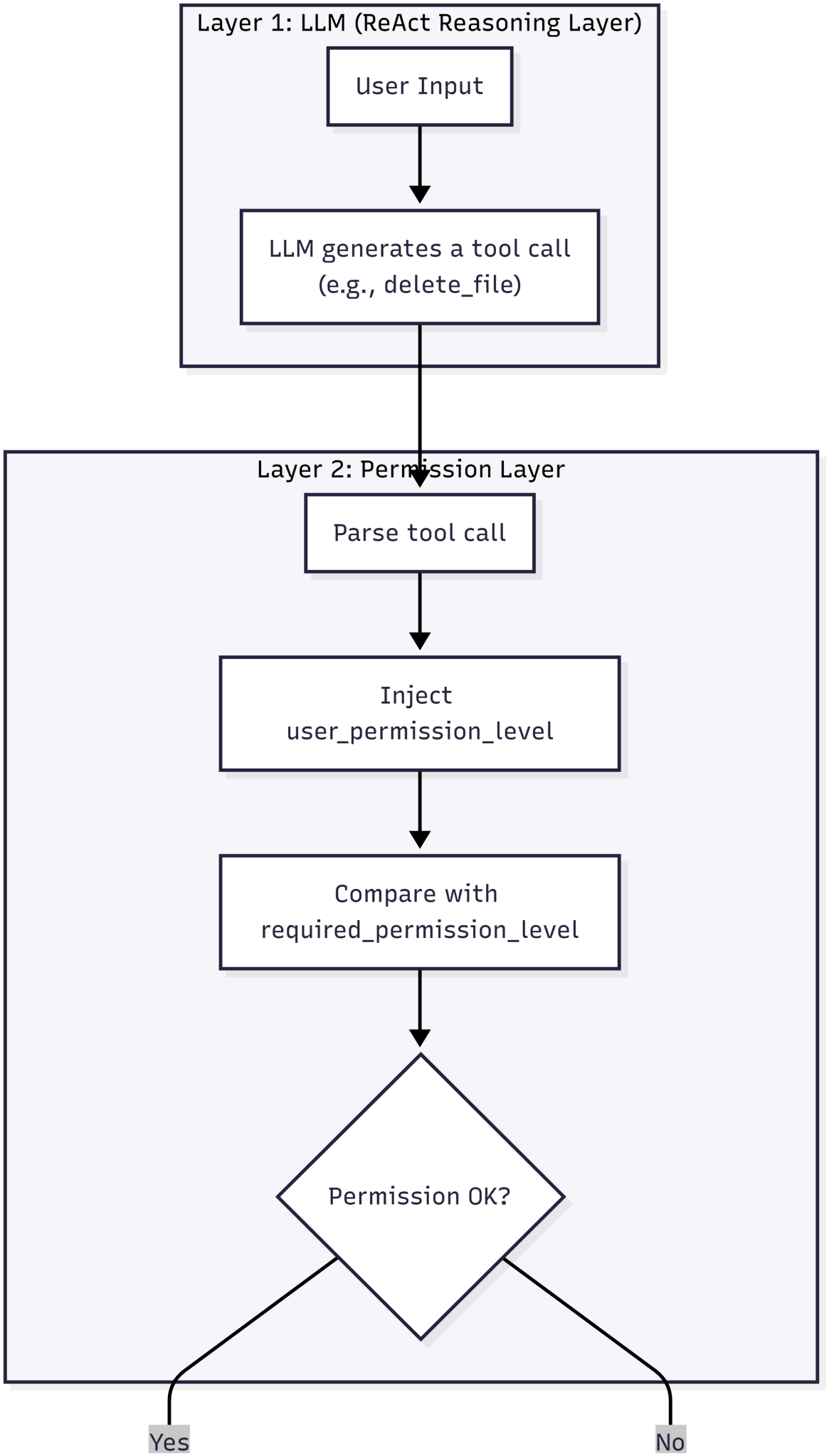
```
AllowedTools = { t | user_permission_level >= t.required_permission_level }
```

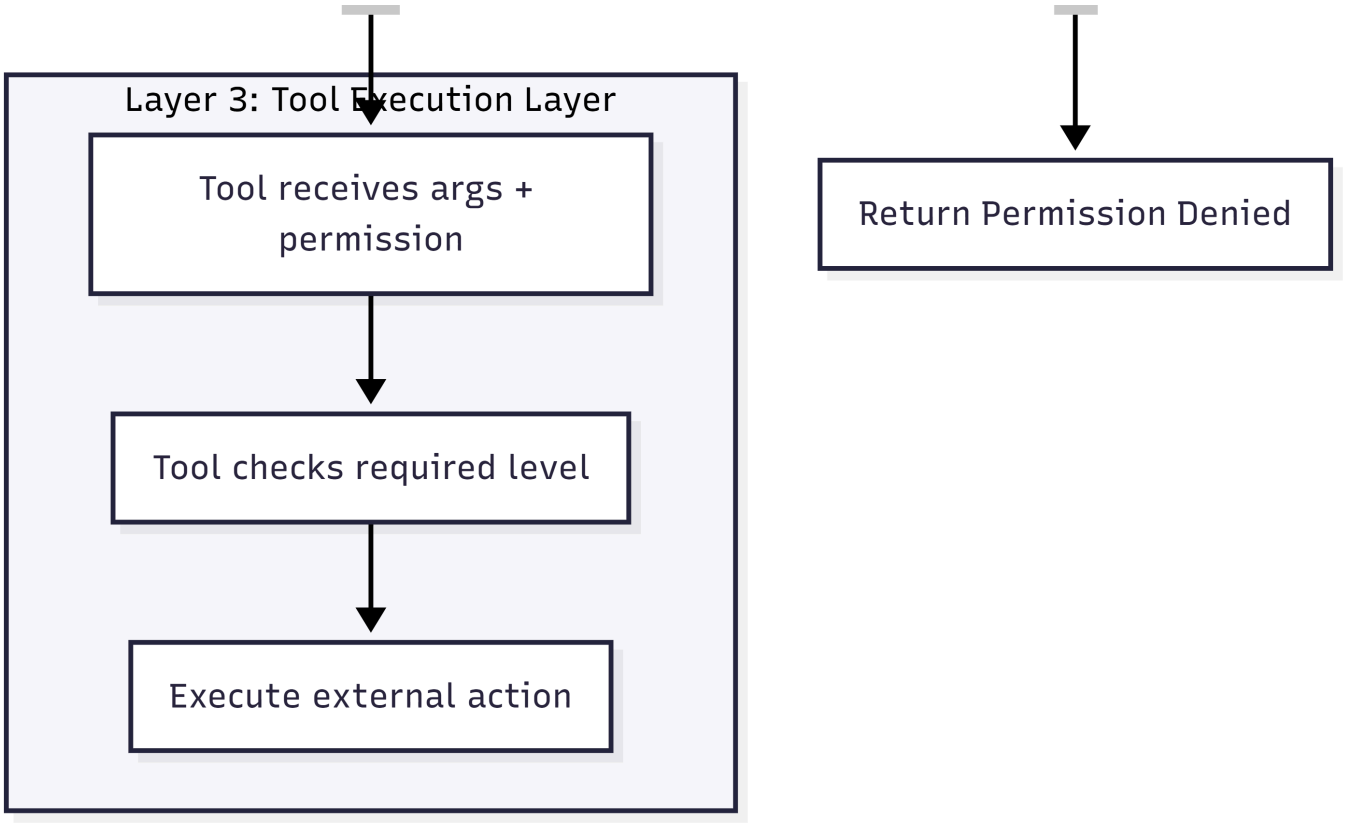
This ensures:

- Incorrect LLM reasoning cannot trigger high-privilege tools
- Jailbreaks cannot execute unauthorized tools
- Least-privilege is automatically enforced
- LLM reasoning remains unrestricted (safety is enforced at the I/O boundary)

5.4 Architecture Diagram




Layer 1: LLM (ReAct Reasoning Layer)
User Input
LLM generates a tool call
(e.g., delete_file)
Layer 2: Permission Layer
Parse tool call
Inject
user_permission_level
Compare with
required_permission_level
Permission OK?
Yes
No



Layer Descriptions

Layer 1: LLM (ReAct Reasoning Layer)

- The LLM performs reasoning and proposes tool calls
- It does **not** generate or modify permission levels
- Tool calls contain only user-provided arguments

Layer 2: Permission Layer

- LangGraph parses tool calls
- Injects the externally provided `user_permission_level`
- Compares it with each tool's `required_permission_level`
- Rejects unauthorized calls before any external action occurs

Layer 3: Tool Execution Layer

- Only authorized calls reach this layer
- Tools receive both arguments and permission levels
- Tools perform the final permission check and execute the action

---

6. Safety Analysis

This section analyzes how the proposed method suppresses external-action risks in ReAct agents.
Rather than constraining the behavior of the LLM, the method ensures safety by
placing the safety boundary at the interface between the LLM and external tools.

6.1 Jailbreak Resistance

In conventional ReAct agents, prompt manipulation (jailbreaking) may cause the LLM to invoke high-privilege tools.
In the proposed method, however, every tool API requires a `user_permission_level`, and
the LLM cannot generate or modify this value.
Therefore, even if the prompt is compromised, unauthorized tools cannot be executed.

Evaluation consists of verifying that:

- the LLM is intentionally given jailbreak prompts
- it attempts to call high-privilege tools
- the Permission Layer consistently rejects these calls

This demonstrates **structural robustness independent of prompt integrity**.

6.2 Hallucination Resistance

LLMs may hallucinate and incorrectly select high-privilege tools.
Under the proposed method,
the Permission Layer always rejects tool calls beyond the user's privilege level,
preventing hallucinations from being converted into external actions.

Evaluation involves:

- giving the LLM ambiguous or misleading instructions (e.g., "delete a non-existent file")
- confirming that high-privilege tool calls are rejected by the Permission Layer

This shows that **reasoning errors do not propagate into external effects**.

6.3 Least Privilege Principle

The proposed method redefines the action space of ReAct as:

```
AllowedTools = { t | user_permission_level >= t.required_permission_level }
```

Thus,
the set of executable tools is automatically restricted based on user privileges,
satisfying the least-privilege principle familiar from RBAC systems.

Evaluation can be performed by comparing:

- tools available to a level-1 user
- tools available to a level-2 user
- tools available to a level-3 user

and confirming that
the action space expands monotonically with privilege level.

6.4 Separation of Concerns

Traditional safety approaches rely on LLM behavior (alignment, guardrails),
but the proposed method delegates safety to **tool-side permission checks**.

This yields several advantages:

- no need to weaken or constrain LLM performance
- reasoning ability remains unrestricted
- safety is enforced at the I/O boundary
- model upgrades do not affect safety guarantees

Evaluation consists of verifying that:

- even if the model is changed (e.g., GPT-4 → GPT-5),
- unauthorized tools remain impossible to execute

This demonstrates **model-independent safety**.

---

7. Implementation Sketch

This section illustrates how the proposed Permission-Aware Tool Interfaces can be integrated into existing ReAct / LangGraph-style agents.
Full implementation details are provided in Appendix A; here we present only the **structural essentials**.

7.1 Example Tool Definition

In the proposed method, every tool must accept `user_permission_level` as an argument and compare it with its own `required_permission_level`:

```
def delete_file(path, user_permission_level):
    required = 3
    if user_permission_level < required:
        return "Permission denied"
    ...
```

Because **permission enforcement occurs inside the tool**,
external actions remain safe even if the LLM behaves incorrectly, drifts, or is jailbroken.

7.2 ReAct Prompt Integration

When initializing the ReAct agent, a **fixed** `user_permission_level` is injected from an external system (e.g., RBAC / IAM).

The LLM is provided with:

- **the user's fixed permission level**
- **the list of available tools**
- **each tool's required permission level**

The LLM proposes tool calls under these constraints,
but **cannot generate or modify the permission value itself**.

7.3 Permission Injection in the ReAct Pipeline

Tool execution in LangGraph / LangChain follows three stages:

1. **The LLM generates a tool-call string**
   Example:

   ```
   delete_file(path="foo.txt")
   ```

2. **LangGraph parses the string into Python arguments**
   → At this stage, **the external `user_permission_level` is injected**

3. **The Python tool function is executed**
   → Permission checks occur inside the tool

Conceptually, the process works as follows:

```
LLM output:
    delete_file(path="foo.txt")

LangGraph parses:
    args = {"path": "foo.txt"}

Permission Layer injects:
    args["user_permission_level"] = agent.permission

Tool executes:
    delete_file(path="foo.txt", user_permission_level=2)
```

Thus,
no matter what the LLM outputs, unauthorized tools cannot be executed,
providing structural safety guarantees.

7.4 Implementation Feasibility

The proposed method is easy to implement for several reasons:

- no modifications to the LLM are required
- the ReAct reasoning structure remains unchanged

- permission injection is added only at the parsing stage
- tools simply add a permission check
- integrates naturally with RBAC / IAM systems
- remains safe across model upgrades (e.g., GPT-4 → GPT-5)

Appendix A includes a **complete implementation** of a Permission-Aware ReAct Agent using ChatOllama (qwen3:1.7b).

---

8. Evaluation

This section describes how the proposed Permission-Aware Tool Interfaces can be evaluated to verify that they structurally suppress external-action risks in ReAct agents.
Because the method enforces safety at the **tool-execution layer**, rather than inside the model,
pass/fail tests are sufficient to validate its guarantees.

8.1 Permission Enforcement Test

Goal:
Verify that **tools beyond the user's privilege level can never be executed**.

Procedure:

1. Instantiate an agent with permission level 1
2. Provide prompts that attempt to invoke tools with `required_permission_level ≥ 2`
3. Confirm that the tool returns `"Permission denied"`

Success Criteria:

- All unauthorized tools are rejected
- Rejection occurs **regardless of the LLM's output content**

8.2 Jailbreak Attempt Test

Goal:
Verify that **the safety boundary cannot be bypassed even under jailbreak conditions**.

Procedure:

1. Provide prompts explicitly designed to induce jailbreak behavior
2. Encourage the LLM to attempt high-privilege tool calls
3. Confirm that the Permission Layer always rejects them

Success Criteria:

- Unauthorized tools remain unexecutable **regardless of jailbreak success**

8.3 Hallucination Stress Test

Goal:
Verify that **LLM hallucinations cannot be converted into external actions**.

Procedure:

1. Provide ambiguous instructions or references to non-existent resources
2. Induce situations where the LLM incorrectly selects high-privilege tools
3. Confirm that the Permission Layer rejects these calls

Success Criteria:

- Incorrect tool selections never result in external actions

8.4 Model-Independence Test

Goal:
Verify that **safety does not depend on the specific LLM used**.

Procedure:

1. Run the same tests across multiple models
   (e.g., qwen3:1.7b → qwen3:7b → GPT-4)
2. Confirm that the Permission Layer consistently rejects unauthorized tools

Success Criteria:

- The safety boundary remains intact across model changes

8.5 Summary of Evaluation Strategy

Because the proposed method is an
architecture that structurally guarantees the safety of external actions,
evaluation only needs to confirm two points:

1. **Unauthorized tools are never executed**
2. **Safety is independent of LLM behavior**
   (jailbreaks, hallucinations, model upgrades)

Full implementation examples and test code are provided in Appendix A.

---

9. Limitations

This section clarifies the limitations of the proposed Permission-Aware Tool Interfaces and delineates the areas that are outside the scope of this work.
While the method provides a structural safety boundary for external actions in ReAct agents, several challenges remain.

9.1 Human-Origin Risks

The method cannot eliminate **human-origin risks (human-in-the-loop risks)**, such as:

- misconfigured permissions
- over-privileged user roles
- RBAC / IAM configuration errors
- operator mistakes

These risks lie outside the Permission Layer and therefore
cannot be mitigated by tool-side permission checks.

9.2 Misconfiguration and Governance

The method assumes that permission levels are correctly configured.
In practice, however, governance issues remain:

- designing appropriate permission granularity
- defining organizational RBAC policies
- updating and revoking permissions
- managing inheritance and delegation

Although the method is compatible with RBAC,
the design of RBAC systems themselves is outside the scope of this work.

9.3 API-Side Vulnerabilities

The Permission Layer protects the **LLM → tool** pathway,
but it cannot protect against vulnerabilities in the APIs or external systems behind the tools.

Examples include:

- authentication bypasses in APIs
- vulnerabilities in external services
- network-level attacks
- OS / filesystem vulnerabilities

These require **separate security measures** beyond the proposed method.

9.4 Permission Leakage

Although the LLM cannot generate or modify `user_permission_level`,
it may still **infer or leak the value**.

Examples:

- responding when asked “What is your permission level?”
- leaking permission information in prompts
- logging or tracing systems exposing permission values

These require implementation-level countermeasures (masking, logging policies)
and **cannot be fully prevented by the method alone**.

9.5 Content Safety Is Out of Scope

This work focuses on **tool-safety (external-action safety)**.
It does **not** address:

- content safety (toxicity, bias, misinformation)
- model-internal alignment
- ethical reasoning or value judgments
- output-quality guarantees

These belong to separate research domains.
The proposed method is specialized for **external-action safety boundaries**.

9.6 Granularity of Permission Levels

The method assumes **tool-level permissions**, but does not explore finer-grained control such as:

- argument-level permissions
- data-access levels
- time- or context-dependent permissions
- dynamic permission inference

These are important directions for future work but are not addressed here.

9.7 Fallback Behavior for Denied Actions

The method does not specify **fallback behavior** when a tool call is denied, such as:

- whether to escalate to a human
- whether to propose alternative tools
- how to design the UI/UX for permission failures

These are important for real-world deployment but
fall outside the scope of this work.

9.8 Summary

The proposed method provides a **structural, model-independent safety boundary**
for external actions in ReAct agents, but its guarantees depend on:

- correct permission configuration
- proper operation of RBAC / IAM systems
- secure APIs and external systems

The method strengthens external-action safety,
but **does not provide complete safety in isolation**.

---

10. Conclusion

This paper proposed an architecture that
structurally suppresses external-action risks in ReAct agents
by introducing permission levels into tool APIs.

Unlike traditional approaches that rely on prompt-based safety or model-internal self-regulation,
the proposed method establishes a **safety boundary at the tool-execution layer**,
eliminating the fundamental vulnerability of ReAct:
unrestricted tool access.

The architecture provides the following properties:

- **Robust to jailbreaks**: unauthorized tools cannot be executed even if prompts are compromised
- **Robust to hallucinations**: incorrect tool selections cannot translate into external actions
- **Least-privilege compliant**: the action space is automatically restricted based on user permissions
- **Model-independent**: safety holds regardless of the LLM's type or capability
- **Easy to implement**: integrates into existing ReAct pipelines by inserting a Permission Layer
- **Compatible with RBAC / IAM**: aligns naturally with real-world permission models

Together, these properties ensure that
the safety of ReAct agents is guaranteed not by the LLM's behavior,
but by the structure of the tool-execution pathway.

This work departs from the conventional assumption that
"LLMs must be weakened to be made safe,"
and instead demonstrates a new design principle:
safety should be enforced at the I/O boundary.
As increasingly powerful models continue to emerge,
this principle provides a practical foundation for achieving both safety and performance.

Future work includes refining permission-level granularity,
exploring dynamic permission inference,
designing fallback behaviors for denied actions,
and integrating the method with broader RBAC governance systems.
Nevertheless, the proposed **Permission-Aware Tool Interfaces** offer a robust and practical foundation
for structurally enhancing the safety of ReAct agents.

Appendix A: Full Permission-Aware ReAct Agent Implementation

Minimal Implement

```
from typing import Dict, Any
from langchain_ollama import ChatOllama
```

```
from langchain_core.globals import set_debug
set_debug(True)

# ----------------------------
# 1. Tool definitions
# ----------------------------

def delete_file(path: str, user_permission_level: int):
    required = 3
    if user_permission_level < required:
        return "Permission denied"
    return f"[Simulated] Deleted file: {path}"

TOOLS = {
    "delete_file": delete_file,
}

TOOL_SPECS = {
    "delete_file": {
        "required_permission_level": 3,
        "args": ["path"]
    }
}

# ----------------------------
# 2. Permission-Aware ReAct Agent
# ----------------------------

class PermissionAwareReActAgent:
    def __init__(self, llm: ChatOllama, tools, tool_specs, user_permission_level:
int):
        self.llm = llm
        self.tools = tools
        self.tool_specs = tool_specs
        self.user_permission_level = user_permission_level

    def run(self, user_input: str):
        prompt = self._build_prompt(user_input)
        llm_output = self.llm.invoke(prompt)

        # AIMessage → string
        llm_text = llm_output.content
        print("LLM Output:", llm_text)

        tool_name, args = self._parse_tool_call(llm_text)
        if tool_name is None:
            return f"LLM Response: {llm_text}"

        # Permission Injection
        args["user_permission_level"] = self.user_permission_level

        return self._execute_tool(tool_name, args)

    # ----------------------------
```

```
    # Prompt construction
    # ----------------------------

    def _build_prompt(self, user_input: str) -> str:
        tool_desc = "\n".join(
            f"- {name}: args={spec['args']}"
            for name, spec in self.tool_specs.items()
        )

        return f"""
            You are a ReAct agent.

            When you decide to use a tool, you MUST output ONLY a Python-style
function call:

                delete_file(path="foo.txt")

            Rules:
            - Output ONLY the function call.
            - Do NOT output explanations.
            - Do NOT wrap the tool name.
            - Do NOT invent new syntax.

            Available tools:
            {tool_desc}

            User input: {user_input}

            Think step-by-step, then output ONLY the tool call.
            """

    # ----------------------------
    # Tool call parsing
    # ----------------------------

    def _parse_tool_call(self, text: str):
        if "(" not in text or ")" not in text:
            return None, {}

        try:
            name = text.split("(")[0].strip()
            arg_str = text.split("(")[1].split(")")[0]

            args = {}
            for pair in arg_str.split(","):
                if "=" in pair:
                    k, v = pair.split("=")
                    args[k.strip()] = v.strip().strip('"')

            return name, args

        except Exception:
            return None, {}
```

```
    # ----------------------------
    # Tool execution
    # ----------------------------

    def _execute_tool(self, tool_name: str, args: Dict[str, Any]):
        if tool_name not in self.tools:
            return f"Unknown tool: {tool_name}"

        tool = self.tools[tool_name]
        return tool(**args)


# ----------------------------
# 3. Instantiate the agent
# ----------------------------

llm = ChatOllama(model="qwen3:1.7b")

agent = PermissionAwareReActAgent(
    llm=llm,
    tools=TOOLS,
    tool_specs=TOOL_SPECS,
    user_permission_level=2
)

# ----------------------------
# 4. Run
# ----------------------------

print(agent.run("Delete the file foo.txt"))
```

Excute Result

```
[llm/start] [llm:ChatOllama] Entering LLM run with input:
{
  "prompts": [
    "Human: \n            You are a ReAct agent.\n\n            When you decide to
use a tool, you MUST output ONLY a Python-style function call:\n\n
delete_file(path=\"foo.txt\")\n\n            Rules:\n            - Output ONLY the
function call.\n            - Do NOT output explanations.\n            - Do NOT
wrap the tool name.\n            - Do NOT invent new syntax.\n\n
Available tools:\n            - delete_file: args=['path']\n\n            User
input: Delete the file foo.txt\n\n            Think step-by-step, then output ONLY
the tool call."
  ]
}
[llm/end] [llm:ChatOllama] [12.38s] Exiting LLM run with output:
{
  "generations": [
```

```
[
  {
    "text": "delete_file(path=\"foo.txt\")",
    "generation_info": {
      "model": "qwen3:1.7b",
      "created_at": "2026-06-13T08:47:51.7466468Z",
      "done": true,
      "done_reason": "stop",
      "total_duration": 12358487900,
      "load_duration": 311166500,
      "prompt_eval_count": 117,
      "prompt_eval_duration": 3454298200,
      "eval_count": 138,
      "eval_duration": 8456697700,
      "logprobs": null,
      "model_name": "qwen3:1.7b",
      "model_provider": "ollama"
    },
    "type": "ChatGeneration",
    "message": {
      "lc": 1,
      "type": "constructor",
      "id": [
        "langchain",
        "schema",
        "messages",
        "AIMessage"
      ],
      "kwargs": {
        "content": "delete_file(path=\"foo.txt\")",
        "response_metadata": {
          "model": "qwen3:1.7b",
          "created_at": "2026-06-13T08:47:51.7466468Z",
          "done": true,
          "done_reason": "stop",
          "total_duration": 12358487900,
          "load_duration": 311166500,
          "prompt_eval_count": 117,
          "prompt_eval_duration": 3454298200,
          "eval_count": 138,
          "eval_duration": 8456697700,
          "logprobs": null,
          "model_name": "qwen3:1.7b",
          "model_provider": "ollama"
        },
        "type": "ai",
        "id": "lc_run--019ec02a-5166-7051-aaeb-aaae8c0d13ce-0",
        "usage_metadata": {
          "input_tokens": 117,
          "output_tokens": 138,
          "total_tokens": 255
        },
        "tool_calls": [],
        "invalid_tool_calls": []
```

```
            }
          }
        }
      ]
    ],
    "llm_output": null,
    "run": null,
    "type": "LLMResult"
  }
  LLM Output: delete_file(path="foo.txt")
  Permission denied
```